

Software Maintenance Exam – What to Expect

Based on what the lecturer explained, the exam will mainly focus on a **reflective report about the lab work we have done during the course**, combined with some theory questions from lectures, slides, and textbooks.

Main Focus of the Exam

The biggest part of the exam is:

- Reflecting on the practical work we did in labs
- Explaining **how** we implemented things
- Explaining **why** we made certain technical decisions
- Connecting our practical work to software maintenance concepts and clean code principles

The lecturer emphasized that the exam is not only about *what* you did, but especially:

- **Why you did it**
 - **How you did it**
-

Exam Format

Reflective Report

We will receive a **report template** during the exam.

The template will contain:

- Different report sections
- Reflection questions
- Questions related to our lab work and code

The report should be based on:

- Our GitHub repository
- The source code we worked on
- The exercises and modifications we made during the course

What We Are Allowed to Bring

We can basically prepare a lot beforehand.

Allowed:

- Textbooks
- Portfolio material
- Lecture notes
- Personal notes/files
- Prewritten templates/drafts
- Diagrams
- Downloaded materials
- Internet access (possibly)

Not allowed:

- Generative AI tools

The lecturer specifically mentioned:

- Disable AI features/modes in browsers/search engines
- Better to rely on downloaded/local material just in case internet access becomes limited

Smart Preparation Strategy

The lecturer strongly hinted that we can prepare a lot in advance.

Recommended preparation:

- Create a **prewritten report template**
- Write notes for every lecture topic
- Prepare explanations for all exercises/labs
- Prepare diagrams beforehand
- Prepare reflections on code modifications

You may even be able to:

- Copy/paste parts from your prepared material into the report
- Rewrite/adapt prepared reflections during the exam

Important Topics to Prepare

Refactoring

Be ready to explain:

- How you refactored code
- Why you refactored it
- Which refactoring techniques/patterns you used

Example question:

- “What is meant by a refactoring pattern?”

Code Smells

You should know:

- How to identify code smells
- Examples of bad code structure
- Why certain code is problematic

Example question:

- “How do you identify a code smell?”

Software Testing

Be prepared to explain:

- How you applied software testing
- Why testing was important
- What kind of tests you used

Continuous Delivery & Pipelines

This seems to be an important topic.

Possible questions:

- How did you create your pipeline?
- How does the pipeline work?
- Why did you structure it that way?

The lecturer specifically mentioned:

- Continuous delivery
 - Pipelines
 - CI/CD-related work
-

Technical Decision-Making

A major focus is explaining your reasoning.

Example:

Instead of only saying:

“I used a switch statement.”

You should explain:

“I used a switch statement instead of multiple if-statements because it improved readability and provided clearer structure for handling multiple conditions.”

The lecturer wants justification behind decisions.

Clean Code & Architecture

The lecturer mentioned:

- Concepts from the *Clean Code* book
- Clean architecture concepts
- Diagrams from Itslearning

You should be able to:

- Apply clean code concepts to your own code

- Explain design decisions
 - Explain how you improved maintainability/readability
-

GitHub Repository Matters

The repository itself will also be evaluated.

This includes:

- Code changes
- Refactoring work
- Testing
- Pipeline setup
- Overall maintenance work

The report should directly reference your repository and source code.

Brain dump from my notes that I used to generate the AI summary:

Software maintenance

Reflective report of the work we have done in our labs how we did it in practice

Will provide a template for report like each section with some questions You can bring all help that you can textbook portfolio, can create files notes whatever, you can use internet but not generative ai anything that can be downloaded like anything you want you can use but not the AI, in the internet disable ai mode before the exam, he doesnt know if we can or cannot use the internet but better to use downloaded stuff

You can create diagrams before hand

You can create a small template/draft basically write the report about every lecture content every single thing and just bring it and rewrite it or just use it

There will be reflection questions about the work we did and then some questions about the content theory from slides or textbook

Example questions

How do you identify a code smell

What is meant by refactoring pattern

Most of the report is reflection of the work but ther will be some questions about course material to have more examination but main report is how you done the work

Github repository will also be evaluated and you base your report on the source code

The grade will also include the repository work what we did with the code basically

Continuous delivery and pipelines

Also you can take lecture notes

Ask for a pipeline how you made it

Final question in some example report he gave was how did you created a pipeline

How did you refactor the code and how did you apply the software testing

Important why you did it and how you did it

Why did you use switch statement instead of if because switch gives more functionalities

Clean code book take some concepts and apply to the individual part and say what kind of decision did you take

The diagram in itslearning about the concepts from clean code architecture book

Can prepare prewritten report and exercise document you can copy paste some stuff from prewritten report

In prewritten report write about exercises and how you have modified the code